

"""

TEST003 - Product API Structure Validation

Validates that the Product Detail API returns the correct structure, field names, and data format required by the frontend and database enrichment pipeline.

"""

```
import pytest
```

```
from mapping.map_enriched_to_product_detail import map_enriched_to_product_detail
```

```
def _sample_product() -> dict:
```

```
    """Full sample enriched product for testing."""
```

```
    return {
```

```
        "barcode": "9310072002678",
```

```
        "productName": "Weet-Bix Cereal",
```

```
        "brand": "Sanitarium",
```

```
        "genericName": "Wholegrain wheat cereal",
```

```
        "additives": ["e330"],
```

```
        "allergens": ["Gluten"],
```

```
        "ingredients": ["wholegrain wheat", "sugar", "salt"],
```

```
        "ingredientsText": "Wholegrain wheat (97%), sugar, salt",
```

```
        "categories": ["Breakfast Cereals"],
```

```
        "labels": ["vegan", "wholegrain"],
```

```
        "nutrientLevels": {"fat": "low", "sugars": "low", "salt": "low"},
```

```
        "nutriments": {
```

```
            "energy-kcal_100g": 354,
```

```
            "energy_100g": 1480,
```

```
            "fat_100g": 1.5,
```

```
            "saturated-fat_100g": 0.3,
```

```
            "carbohydrates_100g": 67.0,
```

```

    "sugars_100g": 3.3,
    "proteins_100g": 12.0,
    "salt_100g": 0.27,
    "sodium_100g": 0.108,
    "fiber_100g": 10.0
  },
  "nutriscoreGrade": "a",
  "productQuantity": 750,
  "productQuantityUnit": "g",
  "servingQuantity": 30,
  "servingQuantityUnit": "g",
  "traces": "Tree nuts",
  "completeness": 0.9,
  "images": {
    "root": "https://images.openfoodfacts.org/images/products/931",
    "primary": "front_en",
    "variants": {"front_en": 5}
  },
  "tags": {"final": ["vegan", "gluten_free"], "removed": []},
  "metadata": {"source": "local-enriched"},
  "enrichmentMetadata": {
    "recommendationScore": 85,
    "reasonTags": ["high_fibre", "low_fat"],
    "similarityMetrics": {}
  }
}

```

— Required fields —————

```
def test_required_field_barcode_present():
```

```
"""Barcode must be present and non-empty."""
```

```
mapped = map_enriched_to_product_detail(_sample_product())
```

```
assert "barcode" in mapped
```

```
assert mapped["barcode"] != ""
```

```
def test_required_field_product_name_present():
```

```
"""Product name must be present and non-empty."""
```

```
mapped = map_enriched_to_product_detail(_sample_product())
```

```
assert "productName" in mapped
```

```
assert mapped["productName"] != ""
```

```
# — Field names match contract —————
```

```
def test_all_contract_fields_present():
```

```
"""All expected contract fields must be present in the response."""
```

```
mapped = map_enriched_to_product_detail(_sample_product())
```

```
expected_fields = [
```

```
    "barcode", "productName", "brand", "genericName",
```

```
    "additives", "allergens", "ingredients", "ingredientsText",
```

```
    "category", "categories", "labels", "nutrientLevels",
```

```
    "nutriments", "nutriments_normalized", "nutriscoreGrade",
```

```
    "productQuantity", "productQuantityUnit", "servingQuantity",
```

```
    "servingQuantityUnit", "traces", "completeness", "images",
```

```
    "tags", "metadata"
```

```
]
```

```
for field in expected_fields:
```

```
    assert field in mapped, f"Missing field: {field}"
```

— Data types are correct —————

```
def test_field_types_are_correct():
    """Field types must match the contract."""
    mapped = map_enriched_to_product_detail(_sample_product())
    assert isinstance(mapped["barcode"], str)
    assert isinstance(mapped["productName"], str)
    assert isinstance(mapped["additives"], list)
    assert isinstance(mapped["allergens"], list)
    assert isinstance(mapped["ingredients"], list)
    assert isinstance(mapped["categories"], list)
    assert isinstance(mapped["labels"], list)
    assert isinstance(mapped["nutrientLevels"], dict)
    assert isinstance(mapped["nutriments"], dict)
    assert isinstance(mapped["nutriments_normalized"], dict)
    assert isinstance(mapped["images"], dict)
    assert isinstance(mapped["tags"], dict)
    assert isinstance(mapped["metadata"], dict)

def test_nutriments_normalized_keys_present():
    """nutriments_normalized must contain all expected keys."""
    mapped = map_enriched_to_product_detail(_sample_product())
    norm = mapped["nutriments_normalized"]
    expected_keys = [
        "energy_kj", "energy_kcal", "fat_g", "saturated_fat_g",
        "carbohydrates_g", "sugars_g", "proteins_g",
        "salt_g", "sodium_mg", "fiber_g"
    ]
    for key in expected_keys:
        assert key in norm, f"Missing nutriments_normalized key: {key}"
```

```
def test_images_has_root_field():
    """Images object must have a root field."""
    mapped = map_enriched_to_product_detail(_sample_product())
    assert "root" in mapped["images"]
    assert mapped["images"]["root"] != ""
```

```
def test_tags_has_final_and_removed():
    """Tags object must have final and removed arrays."""
    mapped = map_enriched_to_product_detail(_sample_product())
    assert "final" in mapped["tags"]
    assert "removed" in mapped["tags"]
    assert isinstance(mapped["tags"]["final"], list)
    assert isinstance(mapped["tags"]["removed"], list)
```

— Missing values handled properly —————

```
def test_missing_brand_returns_none():
    """Missing brand should return None not crash."""
    product = _sample_product()
    product.pop("brand")
    mapped = map_enriched_to_product_detail(product)
    assert mapped["brand"] is None
```

```
def test_missing_allergens_returns_empty_list():
    """Missing allergens should return empty list."""
    product = _sample_product()
```

```
product.pop("allergens")

mapped = map_enriched_to_product_detail(product)

assert mapped["allergens"] == []
```

```
def test_missing_ingredients_returns_empty_list():
    """Missing ingredients should return empty list."""
    product = _sample_product()
    product.pop("ingredients")
    mapped = map_enriched_to_product_detail(product)
    assert mapped["ingredients"] == []
```

```
def test_missing_categories_returns_empty_list():
    """Missing categories should return empty list."""
    product = _sample_product()
    product.pop("categories")
    mapped = map_enriched_to_product_detail(product)
    assert mapped["categories"] == []
```

```
def test_missing_nutriment_returns_empty_dict():
    """Missing nutriment should return empty dict not crash."""
    product = _sample_product()
    product.pop("nutriment")
    mapped = map_enriched_to_product_detail(product)
    assert isinstance(mapped["nutriment"], dict)
```

```
def test_none_traces_handled():
    """None traces should be handled safely."""
```

```
product = _sample_product()
product["traces"] = None
mapped = map_enriched_to_product_detail(product)
assert mapped["traces"] is None
```

— Frontend rendering safety —————

```
def test_response_does_not_crash_on_empty_product():
    """Mapping should not crash even with minimal product data."""
    minimal = {
        "barcode": "9310072002678",
        "productName": "Minimal Product"
    }
    try:
        mapped = map_enriched_to_product_detail(minimal)
        assert mapped["barcode"] != ""
        assert mapped["productName"] == "Minimal Product"
    except Exception as e:
        pytest.fail(f"Mapping crashed on minimal product: {e}")
```

```
def test_array_fields_are_never_none():
    """Array fields must never be None — should be empty list."""
    mapped = map_enriched_to_product_detail({
        "barcode": "9310072002678",
        "productName": "Test"
    })
    array_fields = [
        "additives", "allergens", "ingredients", "categories", "labels"
    ]
```

```
for field in array_fields:

    assert mapped[field] is not None, f"{field} should not be None"

    assert isinstance(mapped[field], list), f"{field} should be a list"
```

— Database enrichment outputs map correctly —————

```
def test_enrichment_metadata_mapped_correctly():

    """enrichmentMetadata should be present when provided."""

    mapped = map_enriched_to_product_detail(_sample_product())

    assert "enrichmentMetadata" in mapped

    assert "recommendationScore" in mapped["enrichmentMetadata"]
```

```
def test_nutriscore_grade_is_valid():

    """nutriscoreGrade must be one of the valid grades."""

    mapped = map_enriched_to_product_detail(_sample_product())

    valid_grades = ["a", "b", "c", "d", "e", "unknown", None]

    assert mapped["nutriscoreGrade"] in valid_grades
```

```
def test_completeness_is_between_0_and_1():

    """Completeness must be between 0 and 1."""

    mapped = map_enriched_to_product_detail(_sample_product())

    if mapped["completeness"] is not None:

        assert 0 <= mapped["completeness"] <= 1
```

```
if __name__ == "__main__":

    product = _sample_product()

    mapped = map_enriched_to_product_detail(product)
```



```
print("TEST003 - Product API Structure Validation")  
print(f"All fields present: {list(mapped.keys())}")  
print("Smoke test passed!")
```